OOP in Java

Complete Interview Preparation Notes

With Examples | Easy Language | Company-Level Questions

Topics Covered	Includes
• Classes & Objects • Encapsulation • Inheritance • Polymorphism • Abstraction • Interfaces	• SOLID Principles • Design Patterns • Java keywords (final, static, super) • Interview Q&A for each topic • Real-world examples • Common mistakes to avoid

Chapter 1: Classes & Objects

What is a Class?

Think of a Class as a blueprint or template. Just like a house blueprint tells you how a house should look, a class tells Java how objects should be made.

Real Life	A 'Car' class is like a car design blueprint. Every actual car (object) is made from that blueprint.

Basic Class Example

```
// Defining a Class
class Car {
    // Attributes (what a car HAS)
    String brand;
    int speed;

    // Method (what a car DOES)
    void drive() {
        System.out.println(brand + " is driving at " + speed + " km/h");
    }
}
```

```
// Creating Objects (instances) from the class
public class Main {
    public static void main(String[] args) {
        Car myCar = new Car();   // Object created!
        myCar.brand = "Toyota";
        myCar.speed = 100;
        myCar.drive();   // Output: Toyota is driving at 100 km/h
    }
}
```

Constructors

A constructor is a special method that runs automatically when you create an object. It sets up the initial values.

```
class Student {
    String name;
    int age;

    // Default Constructor (no arguments)
    Student() {
        name = "Unknown";
        age = 0;
    }

    // Parameterized Constructor
    Student(String n, int a) {
        name = n;
        age = a;
    }

    // Copy Constructor
    Student(Student s) {
        name = s.name;
        age = s.age;
    }
}

// Usage
Student s1 = new Student("Rahul", 20);
Student s2 = new Student(s1);  // Copy of s1
```

Q: What is the difference between a class and an object?
A: A class is a blueprint/template (e.g., Car design). An object is a real instance created from that class (e.g., your actual car). A class is created once; objects can be created many times from it.

Q: Can a constructor be private?
A: Yes! A private constructor prevents object creation from outside the class. This is used in the Singleton Design Pattern where only one object of a class should exist.

Chapter 2: Encapsulation

What is Encapsulation?

Encapsulation means hiding the internal data of a class and only exposing it through safe methods (getters and setters). Think of it like a medicine capsule — the medicine is hidden inside, you can only take it in a controlled way.

Real Life	Your ATM card has a PIN. The bank hides your account details (private). You can only access it through proper methods (getters) — like entering your PIN.

```
class BankAccount {
    private double balance;  // HIDDEN - cannot access directly from outside
    private String owner;

    // Constructor
    BankAccount(String owner, double initialBalance) {
        this.owner = owner;
        this.balance = initialBalance;
    }

    // Getter - controlled way to READ balance
    public double getBalance() {
        return balance;
    }

    // Setter - controlled way to UPDATE balance
    public void deposit(double amount) {
        if (amount > 0) {   // Validation before changing!
            balance += amount;
        }
    }
}

// Usage
BankAccount acc = new BankAccount("Priya", 5000);
// acc.balance = -1000;  ERROR! balance is private
acc.deposit(1000);       // Correct way
System.out.println(acc.getBalance());  // 6000
```

Access Modifiers Quick Reference

Modifier	Same Class	Same Package	Subclass	Everywhere
private	Yes	No	No	No
default	Yes	Yes	No	No

protected	Yes	Yes	Yes	No
public	Yes	Yes	Yes	Yes

Q: Why is encapsulation important?

A: It protects data from being modified accidentally or maliciously. It also lets you add validation rules in setters. Example: You can prevent someone from setting a negative age.

Chapter 3: Inheritance

What is Inheritance?

Inheritance lets one class (child) acquire the properties and methods of another class (parent). This avoids code repetition.

Real Life	A child inherits features from parents — eyes, height, etc. Similarly, a 'Dog' class can inherit from an 'Animal' class — getting eat(), sleep() automatically.

```
// Parent Class
class Animal {
    String name;

    void eat() {
        System.out.println(name + " is eating");
    }

    void sleep() {
        System.out.println(name + " is sleeping");
    }
}

// Child Class - inherits from Animal
class Dog extends Animal {
    void bark() {
        System.out.println(name + " is barking: Woof!");
    }
}

// Usage
Dog d = new Dog();
d.name = "Bruno";
d.eat();    // Inherited from Animal: Bruno is eating
d.bark();   // Dog's own method: Bruno is barking: Woof!
```

Types of Inheritance in Java

Type	Description	Java Support
Single	One child, one parent	Yes
Multilevel	A extends B, B extends C	Yes
Hierarchical	Many children, one parent	Yes
Multiple (via class)	One child, two parents	**NO - not supported**

Multiple (via interface)	One class, two interfaces	Yes

super keyword

```
class Vehicle {
    String brand = 'Generic';
    void show() { System.out.println('I am a vehicle'); }
}

class Car extends Vehicle {
    String brand = 'Toyota';

    void show() {
        super.show();           // Calls parent method
        System.out.println('Brand: ' + super.brand);  // Parent's brand
        System.out.println('My brand: ' + brand);     // My own brand
    }
}
```

Q: Why does Java not support multiple inheritance with classes?

A: To avoid the 'Diamond Problem'. If class C inherits from both A and B, and both A and B have the same method, Java doesn't know which one to use. Java solves this by allowing multiple inheritance only through interfaces.

Q: Can we override a static method?

A: No. Static methods are NOT overridden — they are HIDDEN. Overriding requires runtime polymorphism, but static methods are resolved at compile time.

Chapter 4: Polymorphism

What is Polymorphism?

Polymorphism means 'one name, many forms'. The same method behaves differently based on the object or parameters.

Real Life	A person acts as an employee at work, a father at home, a customer at a shop. Same person, different roles/behaviors!

1. Compile-Time Polymorphism (Method Overloading)

Same method name, different parameters. Decided at COMPILE time.

```
class Calculator {
    // Same name 'add' but different parameters
    int add(int a, int b) {
        return a + b;
    }

    double add(double a, double b) {
        return a + b;
    }

    int add(int a, int b, int c) {
        return a + b + c;
    }
}

Calculator c = new Calculator();
c.add(5, 3);          // Calls first method -> 8
c.add(2.5, 1.5);      // Calls second method -> 4.0
c.add(1, 2, 3);       // Calls third method -> 6
```

2. Runtime Polymorphism (Method Overriding)

Child class provides its own version of a parent method. Decided at RUNTIME based on actual object.

```
class Shape {
    void draw() {
        System.out.println("Drawing a shape");
    }
}

class Circle extends Shape {
    @Override
```

```
    void draw() {
        System.out.println("Drawing a Circle");
    }
}

class Rectangle extends Shape {
    @Override
    void draw() {
        System.out.println("Drawing a Rectangle");
    }
}

// The magic of Runtime Polymorphism!
Shape s;
s = new Circle();       // Shape reference, Circle object
s.draw();               // Output: Drawing a Circle

s = new Rectangle();
s.draw();               // Output: Drawing a Rectangle
```

Q: What is the difference between overloading and overriding?

A: Overloading: same class, same method name, different parameters. Resolved at COMPILE time. Overriding: parent and child class, same method name + parameters. Resolved at RUNTIME. Overriding uses @Override annotation.

Q: Can we override private methods?

A: No. Private methods are not visible to child classes, so they cannot be overridden. The child class can define a method with the same name, but it is a new method, not an override.

Chapter 5: Abstraction

What is Abstraction?

Abstraction means hiding the implementation details and only showing what is necessary. You use a phone without knowing how circuits work — that's abstraction!

1. Abstract Classes

```
// Abstract class - cannot be instantiated directly
abstract class Vehicle {
    String brand;

    // Abstract method - no body, MUST be implemented by child
    abstract void start();

    // Normal method - already has body
    void stop() {
        System.out.println("Vehicle stopped");
    }
}

class Car extends Vehicle {
    @Override
    void start() {
        System.out.println("Car starts with key");
    }
}

class Bike extends Vehicle {
    @Override
    void start() {
        System.out.println("Bike starts with kick");
    }
}

// Vehicle v = new Vehicle();  ERROR - cannot instantiate abstract class
Car c = new Car();
c.start();   // Car starts with key
c.stop();    // Vehicle stopped (inherited)
```

2. Interfaces (100% Abstraction)

```
// Interface - all methods are abstract by default
interface Payable {
    void pay(double amount);   // abstract by default
```

```
    void getReceipt();          // abstract by default
}

// A class can implement MULTIPLE interfaces
interface Refundable {
    void refund(double amount);
}

class OnlinePayment implements Payable, Refundable {
    @Override
    public void pay(double amount) {
        System.out.println("Paid Rs." + amount + " online");
    }

    @Override
    public void getReceipt() {
        System.out.println("Receipt sent to email");
    }

    @Override
    public void refund(double amount) {
        System.out.println("Refunded Rs." + amount);
    }
}
```

Abstract Class vs Interface - Key Differences

Feature	Abstract Class	Interface
Variables	Any type	public static final only
Methods	Abstract + concrete	Abstract (+ default since Java 8)
Constructor	Can have	Cannot have
Inheritance	extends (single)	implements (multiple)
When to use	Shared code + template	Define contract/behavior

Q: Can an abstract class have a constructor?
A: Yes! Even though you cannot create an object of an abstract class directly, the constructor is called when a child class object is created (via super()).

Chapter 6: Important Java Keywords

final keyword

final can be applied to variables, methods, and classes — each has a different meaning.

```
// 1. final VARIABLE - value cannot change (constant)
final int MAX_SPEED = 120;
// MAX_SPEED = 150;  ERROR!

// 2. final METHOD - cannot be overridden in child class
class Parent {
    final void show() {
        System.out.println("Cannot be changed");
    }
}

// 3. final CLASS - cannot be inherited
final class Secure {  // No class can extend Secure
    // Example: String class in Java is final!
}
```

static keyword

```
class Counter {
    static int count = 0;  // Shared by ALL objects
    String name;

    Counter(String n) {
        name = n;
        count++;  // Every time an object is created, count increases
    }

    static void showCount() {
        System.out.println("Total objects: " + count);
    }
}

Counter c1 = new Counter("Amit");
Counter c2 = new Counter("Priya");
Counter c3 = new Counter("Ravi");
Counter.showCount();  // Total objects: 3
// Static method called on CLASS name, not object
```

this keyword

```java
class Person {
    String name;
    int age;

    // 'this' distinguishes field from parameter
    Person(String name, int age) {
        this.name = name;  // 'this.name' = field, 'name' = parameter
        this.age = age;
    }

    // 'this()' calls another constructor
    Person(String name) {
        this(name, 18);  // Calls the above constructor
    }
}
```

Q: What is the difference between static and instance methods?

A: Static methods belong to the CLASS — called via ClassName.method(). They cannot access instance variables. Instance methods belong to an OBJECT — called via object.method(). They can access all instance and static members.

Chapter 7: SOLID Principles

SOLID principles are 5 design principles that make your code clean, maintainable, and interview-friendly!

Letter	Principle	Meaning	Example
S	Single Responsibility	One class = one job only	Invoice class only handles invoice, not printing
O	Open/Closed	Open for extension, closed for modification	Add new payment type without changing existing code
L	Liskov Substitution	Child class should work in place of parent	If Bird has fly(), Penguin (can't fly) should not extend it directly
I	Interface Segregation	Don't force classes to implement unused methods	Separate Printable and Scannable interfaces instead of one big one
D	Dependency Inversion	Depend on abstractions, not concrete classes	Use interface types in method parameters, not specific implementations

Chapter 8: Common Design Patterns

1. Singleton Pattern

Ensures only ONE object of a class is ever created. Used for: Database connections, Logger, Config settings.

```
class DatabaseConnection {
    private static DatabaseConnection instance; // Only one!

    private DatabaseConnection() {}  // Private constructor

    public static DatabaseConnection getInstance() {
        if (instance == null) {
            instance = new DatabaseConnection();
        }
        return instance;
    }
}

// Usage - always returns SAME object
DatabaseConnection db1 = DatabaseConnection.getInstance();
DatabaseConnection db2 = DatabaseConnection.getInstance();
// db1 == db2 -> TRUE! Same object!
```

2. Factory Pattern

Creates objects without specifying the exact class. Used when you don't know which class to create at compile time.

```
interface Animal {
    void speak();
}

class Dog implements Animal {
    public void speak() { System.out.println("Woof!"); }
}

class Cat implements Animal {
    public void speak() { System.out.println("Meow!"); }
}

// Factory - decides which object to create
class AnimalFactory {
    public static Animal create(String type) {
        if (type.equals("dog")) return new Dog();
        if (type.equals("cat")) return new Cat();
```

```
        return null;
    }
}

// Usage
Animal a = AnimalFactory.create("dog");
a.speak();  // Woof!
```

Quick Revision: Most Asked Interview Q&A

Q: What are the 4 pillars of OOP?

A: 1. Encapsulation - hiding data. 2. Inheritance - reusing code from parent. 3. Polymorphism - one name, many forms. 4. Abstraction - hiding implementation, showing only essentials.

Q: What is the difference between abstract class and interface?

A: Abstract class: can have constructors, instance variables, concrete methods. Supports single inheritance. Interface: no constructors, all variables are static final. Supports multiple implementation. Use abstract class for IS-A relationship with shared code; use interface for HAS-A behavior (contract).

Q: What is dynamic dispatch?

A: Dynamic dispatch (runtime polymorphism) means the method that gets called is decided at RUNTIME based on the actual object type, not the reference type. Example: Shape s = new Circle(); s.draw() calls Circle's draw(), not Shape's.

Q: What is the difference between composition and inheritance?

A: Inheritance = IS-A (Dog IS-A Animal). Composition = HAS-A (Car HAS-A Engine). Prefer composition because it's more flexible and avoids tight coupling. With composition, you can change behavior at runtime.

Q: Can we have a constructor in an interface?

A: No. Interfaces cannot be instantiated, so constructors don't make sense. In Java 8+, interfaces can have default and static methods with bodies, but no constructors.

Q: What is method hiding vs method overriding?

A: Method overriding: instance methods in child class override parent methods. Resolved at runtime. Method hiding: static methods in child class HIDE parent static methods. Resolved at compile time. @Override annotation does NOT apply to static methods.

Final Tips for Your Interview

- Always explain with a real-world example — interviewers love it
- Know the WHY — why use interface over abstract class, why encapsulation, etc.
- Practice writing code by hand — companies test on paper/whiteboard
- SOLID principles are asked in mid-level company interviews especially
- Singleton and Factory patterns come up very frequently — know them cold